

problem_2

October 17, 2025

```
[1]: import os
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import numpy as np
import time
import matplotlib.pyplot as plt
from tqdm import trange

from bound_propagation import BoundModelFactory, HyperRectangle
```

```
[2]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

0.0.1 Basic setup

```
[3]: # simple nn model
class Normalize(nn.Module):
    def forward(self, x):
        return (x - 0.1307)/0.3081
class SimpleNNModel(nn.Sequential):
    def __init__(self):
        super().__init__(
            # Normalize(),
            nn.Linear(784, 50),
            nn.ReLU(),
            nn.Linear(50, 50),
            nn.ReLU(),
            nn.Linear(50, 10),
        )
```

```
[4]: def construct_transform():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(0.1307, 0.3081),
```

```

        transforms.Lambda(torch.flatten)
    ])

    # Identity transform - because cross entropy loss supports class indexing
    target_transform = transforms.Compose([])

    return transform, target_transform

```

0.0.2 Standard training

```

[5]: def train(net, epochs):
    print(' [TRAINING] ')

    transform, target_transform = construct_transform()
    train_data = datasets.FashionMNIST('../hw2/mnist_data', train=True,
    ↪download=True,
                                     transform=transform,
    ↪target_transform=target_transform)
    train_loader = DataLoader(train_data, batch_size=500, shuffle=True,
    ↪num_workers=8)

    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(net.parameters(), lr=5e-4)

    pbar = trange(epochs)
    for epoch in pbar:

        running_loss = 0.0
        for i, (X, y) in enumerate(train_loader):
            X, y = X.to(device), y.to(device)
            optimizer.zero_grad(set_to_none=True)
            y_hat = net(X)
            loss = criterion(y_hat, y)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            pbar.set_postfix(epoch=epoch+1, loss=running_loss)

```

0.0.3 Define adversarial training, logits and loss functions

```

[6]: def adversarial_logit(y_hat, y):
    batch_size = y.size(0)
    classes = torch.arange(10, device=y.device).unsqueeze(0).expand(batch_size,
    ↪-1)
    mask = (classes == y.unsqueeze(-1)).to(dtype=y_hat.lower.dtype)

```

```

    # Take upper bound for logit of all but the correct class where you take
    ↪ the lower bound
    adversarial_logit = (1 - mask) * y_hat.upper + mask * y_hat.lower

    return adversarial_logit

def adversarial_prob_margin(y_hat, y):
    batch_size = y.size(0)
    y_index = (torch.arange(batch_size, device=y.device), y)

    logit = adversarial_logit(y_hat, y)

    probs = F.softmax(logit, dim=1)
    label_probs = probs.gather(1, y.unsqueeze(1))

    others_mask = torch.ones_like(probs, dtype=torch.bool)
    others_mask[y_index] = False
    others_probs = probs[others_mask].view(batch_size, -1)

    return torch.min(label_probs - others_probs, dim=1).values

```

```

[7]: def ibp_train(net, epochs):
    print('[ROBUST TRAINING]')

    transform, target_transform = construct_transform()
    train_data = datasets.FashionMNIST('../hw2/mnist_data', train=True,
    ↪ download=True,
                                transform=transform,
    ↪ target_transform=target_transform)
    train_loader = DataLoader(train_data, batch_size=500, shuffle=True,
    ↪ num_workers=8)

    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(net.parameters(), lr=5e-4)

    k = 1.0
    eps_train = 0
    pbar = trange(epochs)
    for epoch in pbar:

        running_loss = 0.0
        running_cross_entropy = 0.0
        for i, (X, y) in enumerate(train_loader):
            X, y = X.to(device), y.to(device)
            optimizer.zero_grad(set_to_none=True)

```

```

        y_hat = net(X)

        cross_entropy = criterion(y_hat, y)

        bounds = net.ibp(HyperRectangle.from_eps(X, eps_train))
        logit = adversarial_logit(bounds, y)

        loss = k * cross_entropy + (1 - k) * criterion(logit, y)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        running_cross_entropy += cross_entropy.item()
        pbar.set_postfix(epoch=epoch, ce=running_cross_entropy, loss =
↪running_loss)

    eps_train = min(eps_train + 0.01, 0.1)
    k = max(k - 0.05, 0.5)

```

```

[8]: def pgd_linf_untargeted(model, x, labels, k, eps, eps_step):
    model.eval()
    ce_loss = torch.nn.CrossEntropyLoss()
    adv_x = x.clone().detach()
    adv_x.requires_grad_(True)
    for _ in range(k):
        adv_x.requires_grad_(True)
        model.zero_grad()
        output = model(adv_x)
        # TODO: Calculate the loss
        loss = ce_loss(output, labels)
        loss.backward()
        with torch.no_grad():
            # TODO: compute the adv_x
            adv_x = adv_x + (eps_step * adv_x.grad.sign())
            # find delta, clamp with eps
            delta = adv_x - x
            delta = torch.clamp(delta, -eps, eps)
            adv_x = torch.clamp(x + delta, 0, 1)

    return adv_x

```

```

[9]: @torch.no_grad()
def test(net):
    print(' [TEST] ')

    transform, target_transform = construct_transform()

```

```

    test_data = datasets.FashionMNIST('../hw2/mnist_data', train=False,
↪download=True,
                                transform=transform,
↪target_transform=target_transform)
    test_loader = DataLoader(test_data, batch_size=100, shuffle=False,
↪num_workers=8)

    correct = 0
    for i, (X, y) in enumerate(test_loader):
        X, y = X.to(device), y.to(device)

        y_hat = net(X)

        predicted = torch.argmax(y_hat, 1)
        correct += (predicted == y).sum().item()

    print(f'Standard Accuracy: {correct / len(test_data):.3f}')

```

```

[10]: def robustness_test(net, eps):
    print(f'[ROBUSTNESS TEST] eps: {eps}')

    transform, target_transform = construct_transform()
    test_data = datasets.FashionMNIST('../hw2/mnist_data', train=False,
↪download=True,
                                transform=transform,
↪target_transform=target_transform)
    test_loader = DataLoader(test_data, batch_size=100, shuffle=False,
↪num_workers=8)

    robust_correct = 0
    for i, (X, y) in enumerate(test_loader):
        X, y = X.to(device), y.to(device)

        adv_x = pgd_linf_untargeted(net, X, y, 10, eps, eps/4)
        adv_y_hat = net(adv_x)
        predicted = torch.argmax(adv_y_hat, 1)
        robust_correct += (predicted == y).sum().item()

    print(f'Robust Accuracy: {robust_correct / len(test_data):.3f}')

```

0.0.4 Define and train standard and robust models

```

[11]: # define standard model
standard_net = SimpleNNModel().to(device)

```

```
[12]: # train standard model
start_time = time.time()
train(standard_net, epochs=20)
print(f"Standard training time: {time.time() - start_time:.2f} seconds")
```

[TRAINING]

100%| | 20/20 [00:21<00:00, 1.09s/it, epoch=20, loss=34.1]

Standard training time: 21.97 seconds

```
[13]: # test standard model
standard_net.eval()
test(standard_net)
```

[TEST]

Standard Accuracy: 0.875

```
[14]: # define robust model
robust_net = SimpleNNModel().to(device)
factory = BoundModelFactory()
bounded_net = factory.build(robust_net)
```

```
[15]: # train robust model
start_time = time.time()
bounded_net.train()
ibp_train(bounded_net, epochs=20)
print(f"Robust training time: {time.time() - start_time:.2f} seconds")
```

[ROBUST TRAINING]

100%| | 20/20 [00:23<00:00, 1.18s/it, ce=53.6, epoch=19, loss=72.8]

Robust training time: 23.74 seconds

```
[16]: # test robust model
bounded_net.eval()
test(bounded_net)
```

[TEST]

Standard Accuracy: 0.830

```
[18]: for eps in np.arange(0.01, 0.11, 0.01):
robustness_test(bounded_net, eps)
```

[ROBUSTNESS TEST] eps: 0.01

Robust Accuracy: 0.779

[ROBUSTNESS TEST] eps: 0.02

Robust Accuracy: 0.776

[ROBUSTNESS TEST] eps: 0.03

```

Robust Accuracy: 0.774
[ROBUSTNESS TEST] eps: 0.04
Robust Accuracy: 0.770
[ROBUSTNESS TEST] eps: 0.05
Robust Accuracy: 0.767
[ROBUSTNESS TEST] eps: 0.060000000000000005
Robust Accuracy: 0.763
[ROBUSTNESS TEST] eps: 0.06999999999999999
Robust Accuracy: 0.759
[ROBUSTNESS TEST] eps: 0.08
Robust Accuracy: 0.756
[ROBUSTNESS TEST] eps: 0.09
Robust Accuracy: 0.753
[ROBUSTNESS TEST] eps: 0.09999999999999999
Robust Accuracy: 0.749

```

```

[19]: for eps in np.arange(0.01, 0.11, 0.01):
        robustness_test(standard_net, eps)

```

```

[ROBUSTNESS TEST] eps: 0.01
Robust Accuracy: 0.765
[ROBUSTNESS TEST] eps: 0.02
Robust Accuracy: 0.750
[ROBUSTNESS TEST] eps: 0.03
Robust Accuracy: 0.739
[ROBUSTNESS TEST] eps: 0.04
Robust Accuracy: 0.725
[ROBUSTNESS TEST] eps: 0.05
Robust Accuracy: 0.712
[ROBUSTNESS TEST] eps: 0.060000000000000005
Robust Accuracy: 0.700
[ROBUSTNESS TEST] eps: 0.06999999999999999
Robust Accuracy: 0.687
[ROBUSTNESS TEST] eps: 0.08
Robust Accuracy: 0.677
[ROBUSTNESS TEST] eps: 0.09
Robust Accuracy: 0.667
[ROBUSTNESS TEST] eps: 0.09999999999999999
Robust Accuracy: 0.658

```

```

[ ]:

```